

Creeping Crawler

Lapo Siciliani
2007890

Fabio Priori
1938446

Tamer Hayek
1897438

1 Obiettivo del Progetto

Creeping Crawler è una pipeline end-to-end per l’acquisizione e l’analisi di contenuti testuali da sorgenti web eterogenee, progettata per alimentare sistemi RAG e modelli linguistici. Il sistema scarica pagine web tramite Crawl4AI, le trasforma in testo pulito tramite parser specifici per dominio, ne valuta la qualità di estrazione confrontando l’output con un Gold Standard costruito manualmente (metriche quantitative e giudizio LLM), e persiste ogni dato in un database relazionale. Tutte le funzionalità sono esposte via API REST e da un’interfaccia web.

2 Architettura

Il sistema è composto da **quattro container Docker** orchestrati da `docker-compose.yml`:

- **backend** — server FastAPI con crawler, parser, valutazione e API REST (porta 8003);
- **frontend** — server FastAPI + Jinja2 per la Web UI (porta 8004);
- **database** — MariaDB con i dati persistenti (porta 3306);
- **ollama** — server LLM locale per il giudice (porta 11434).

All’avvio il backend inizializza il pool di connessioni, popola il DB dai Gold Standard (se vuoto) e pre-calcola le metriche quantitative, così che il sistema sia subito interrogabile senza passaggi manuali.

3 Parser e Acquisizione

Il modulo `src/lib/crawling/` è basato su **Crawl4AI** (browser Chromium headless, avviato una sola volta e riusato). `fetch_page` esegue il crawl live con la configurazione del dominio; `fetch_page_from_html` riparsa un HTML

già salvato tramite lo schema `raw:`, senza browser. La sottocartella `domains/` contiene un `CrawlerRunConfig` per ciascun dominio (esclusione di tag e selettori CSS non pertinenti; modalità `magic` per la gestione automatica dei blocchi anti-bot).

Il modulo `src/lib/parsers/` definisce l’interfaccia astratta `ContentParser` con implementazioni per Wikipedia, ESPN, CNBC e XE, più un `PassThroughParser` di fallback per i domini non registrati. Ogni parser opera sul Markdown prodotto da Crawl4AI: interrompe la raccolta alle sezioni boilerplate e rimuove pattern residui tramite espressioni regolari.

4 Gold Standard e Database

Colonna	Tipo	Chiave
<i>web_resources</i> (obbligatoria)		
url	varchar(2048)	PK
domain	varchar(255)	
title	varchar(2048)	
html_text	longtext	
created_at	datetime	
<i>gold_standard</i> (obbligatoria)		
url	varchar(2048)	PK, FK
gold_text	longtext	
created_at	datetime	
<i>evaluations</i> (aggiuntiva, non richiesta)		
url	varchar(2048)	PK, FK
precision_val ... excess_ratio	double	
judge_score	int	
updated_at	datetime	

Table 1: Schema del database. Le chiavi esterne di `gold_standard` e `evaluations` riferenziano `web_resources.url` (ON DELETE CASCADE).

Il Gold Standard è costruito manualmente e salvato come file JSON in `gs_data/` (uno per dominio), che fungono da seed. Il DBMS è **MariaDB**. Lo schema (definito in `init.sql` ed esposto da `GET /db_schema`) è organizzato in tre tabelle (Tabella 1): `web_resources` conserva l’HTML

grezzo, gold_standard il testo di riferimento in relazione 1-a-1, ed evaluations le metriche e il judge_score. Solo le prime due sono richieste dalla specifica; evaluations è una tabella aggiuntiva che abbiamo introdotto come archivio dei risultati.

Al primo avvio uno script di inizializzazione idempotente carica automaticamente i JSON nel DB (HTML grezzo e gold_text), rendendo il parsing rieseguibile offline e le valutazioni immediatamente disponibili.

5 Valutazione dei Parser

Metodologia. Ogni parser viene valutato su 10 URL per dominio tramite GET /full_gs_eval, che usa l'HTML statico salvato nel DB (nessun crawl live) per garantire ripetibilità. Il testo estratto viene passato per strip_markdown prima del confronto, così da normalizzare quote tipografiche, trattini Unicode ed entità HTML. Le metriche set-based (obbligatoria token_level_eval) operano su insiemi di token unici; le metriche vettoriali (cosine, Jaccard, excess ratio) operano sui conteggi di frequenza tramite Counter.

Risultati. Le tabelle riportano i valori medi su tutto il Gold Standard per ciascun dominio.

Dominio	Precision	Recall	F1
it.wikipedia.org	0.9832	0.9774	0.9802
www.espn.com	0.9993	0.9983	0.9988
www.cnn.com	0.9988	0.9999	0.9993
www.xe.com	0.9871	0.9849	0.9858

Table 2: Token Level Eval — metriche set-based medie per dominio.

Dominio	Cosine	Jaccard	Excess Ratio
it.wikipedia.org	0.9949	0.9617	0.0115
www.espn.com	0.9802	0.9977	0.0185
www.cnn.com	1.0000	0.9986	0.0006
www.xe.com	0.9924	0.9725	0.0253

Table 3: Similarity Eval — metriche vettoriali medie per dominio (Excess Ratio: più basso è meglio).

Nota su www.espn.com — antibot e fallback al DB. ESPN ha aggiornato le proprie protezioni anti-bot: il crawl live restituisce ora una pagina di blocco invece dell'articolo. Il sistema gestisce esplicitamente questo caso: quando il crawl live fallisce, l'endpoint /parse ricade automaticamente sull'HTML salvato nel DB (segna-

lando fallback_used). La valutazione, che usa comunque sempre l'HTML statico del DB, non è quindi influenzata dal blocco.

6 LLM-as-Judge

Oltre alle metriche quantitative, un giudice LLM valuta la qualità del testo estratto. Il modello gira in locale tramite **Ollama**: tra i cinque modelli candidati proposti (gemini, llama3.2:3b, phi4-mini, qwen3:4b, mistral) è stato scelto llama3.2:3b, buon compromesso tra qualità del giudizio e velocità su CPU. Testo estratto e Gold Standard (ripuliti dal Markdown e troncati a 1000 caratteri per contenere i tempi) sono inviati in un prompt strutturato che richiede una risposta JSON con feedback e score intero da 1 a 5. Il formato è forzato tramite constrained decoding; è comunque previsto un fallback robusto (score minimo e feedback diagnostico) qualora la risposta non sia interpretabile.

Dominio	Judge Score
it.wikipedia.org	4.60
www.espn.com	4.50
www.cnn.com	4.30
www.xe.com	4.40

Table 4: Judge Eval — punteggio medio per dominio assegnato dal giudice LLM (scala 1–5).

7 API REST

Il backend FastAPI espone gli endpoint richiesti dalla specifica, con validazione I/O tramite Pydantic e gestione esplicita degli errori (dominio non supportato, URL irraggiungibile o assente nel DB):

- POST /parse
Parsing di un URL. Body: {url, local?}; local sceglie tra copia nel DB e crawl live (con fallback al DB).
- GET /domains
Domini supportati. Nessun parametro.
- GET /gold_standard
Entry del GS per un URL. Query: url.
- GET /gold_standard_urls
URL del GS per un dominio. Query: domain.
- POST /evaluate
Metriche quantitative. Body: {parsed_text, gold_text}.

- **POST /evaluate_judge**
Giudizio LLM. Body: {parsed_text, gold_text}.
- **GET /full_gs_eval**
Media aggregata (metriche + judge_score) su un dominio. Query: domain.
- **POST /add_web_resource**
Inserisce una risorsa web. Body: {url, html_text}.
- **POST /add_gold_standard**
Inserisce un'entry del GS. Body: {url, gold_text}.
- **DELETE /web_resource**
Rimuove una risorsa, in cascata sul GS collegato. Body: {url}.
- **DELETE /gold_standard**
Rimuove solo l'entry del GS. Body: {url}.
- **GET /db_stats**
Conteggi e medie per dominio. Nessun parametro.
- **GET /db_schema**
Schema del DB. Nessun parametro.
- **GET /status**
Salute di backend, database e Ollama (sempre HTTP 200; lo stato è nel corpo JSON). Nessun parametro.

8 Web UI

Il frontend (FastAPI + Jinja2, HTML/CSS) permette di usare l'intera pipeline senza toccare direttamente le API: tutta la logica passa esclusivamente per le REST del backend, e gli errori restituiti sono mostrati chiaramente. È composto da una home page (navigazione, matricole del gruppo, stato del sistema e domini supportati) e da tre sezioni:

- **Parser & Evaluation** — testa un parser su un URL (modalità live/local), mostra HTML grezzo e testo pulito e, se l'URL è nel GS, tutte le metriche e il giudizio LLM;
- **Gold Standard Builder** — costruisce e gestisce il GS nel DB (scarica l'HTML, inserisce il gold_text, elenca ed elimina le entry esistenti);
- **Stats** — panoramica aggregata per dominio: conteggi delle tabelle, metriche medie e judge score medio.

9 Orchestrazione e Robustezza

Automazione con Makefile. Un Makefile centralizza i comandi ricorrenti: setup degli ambienti e delle dipendenze, avvio in locale, gestione dello stack Docker (up, down, logs), un target reset che ricrea l'intero stack da zero, e un target test che esegue il grader ufficiale.

Healthcheck e avvio ordinato. MariaDB e Ollama espongono un healthcheck e diventano *healthy* solo quando sono realmente pronti: MariaDB quando accetta connessioni, Ollama dopo il download del modello llama3.2:3b. Il backend si avvia solo a quel punto, grazie ai depends_on con condition: service_healthy, e il frontend parte a sua volta dopo il backend. L'avvio è quindi ordinato e il sistema è utilizzabile solo quando tutte le dipendenze lo sono, gestendo anche il primo boot in cui il pull del modello richiede alcuni minuti.

Extra

Similarity Eval. Oltre alla token_level_eval obbligatoria, un secondo gruppo di metriche su vettori di frequenza (*cosine*, *Jaccard*, *excess ratio*) rileva contenuto ridondante non catturato da precision/recall.

Monaco Diff Editor. La pagina Parser & Evaluation integra Monaco Editor in modalità diff affiancato, evidenziando le differenze tra testo estratto e Gold Standard (e tra i testi inviati al giudice) per guidare il tuning dei parser.

Overlay di caricamento. Alcune operazioni (parsing live, valutazione e soprattutto il giudizio LLM) possono richiedere diversi secondi. Per non far percepire l'interfaccia come bloccata, la Web UI mostra un overlay di caricamento all'invio dei form, che migliora l'esperienza utente e previene invii multipli accidentali.

Pre-calcolo all'avvio. All'avvio del backend vengono pre-calcolate, a partire dall'HTML nel DB, le sole metriche quantitative, escludendo il giudizio LLM: valutare il judge su tutto il Gold Standard richiederebbe oltre 10 minuti. Il judge_score viene quindi calcolato on demand da /full_gs_eval, che ne memorizza il risultato.